FLUTTER STATE MANAGEMENT

Riverpod Revision & Bloc / Cubit

Lecture 6: a quick recap of Riverpod 3.x, then event-driven state management with flutter_bloc: Cubit, Bloc, stream-based transformations, functional state isolation and clean architecture layers

OVERVIEW

Agenda

1 Riverpod revision

Providers, Ref, Notifiers, AsyncValue, 3.x, codegen (15 slides)

2 Bloc foundations

Why Bloc, the BLoC pattern, packages, Streams primer

3 Cubit

Simple state with emit(), rich state classes, async loading

4 Bloc and events

Sealed events and states, on<Event>, Emitter, event transformers

5 flutter_bloc widgets

BlocProvider, BlocBuilder, BlocListener, BlocConsumer, context.read

6 Stream transformations

Debounce, switchMap, emit.forEach, optimistic updates

7 Clean architecture

Layers, repositories, DI, functional state isolation

8 Testing and comparison

bloc_test, widget tests, Riverpod vs Bloc decision guide

PART 1

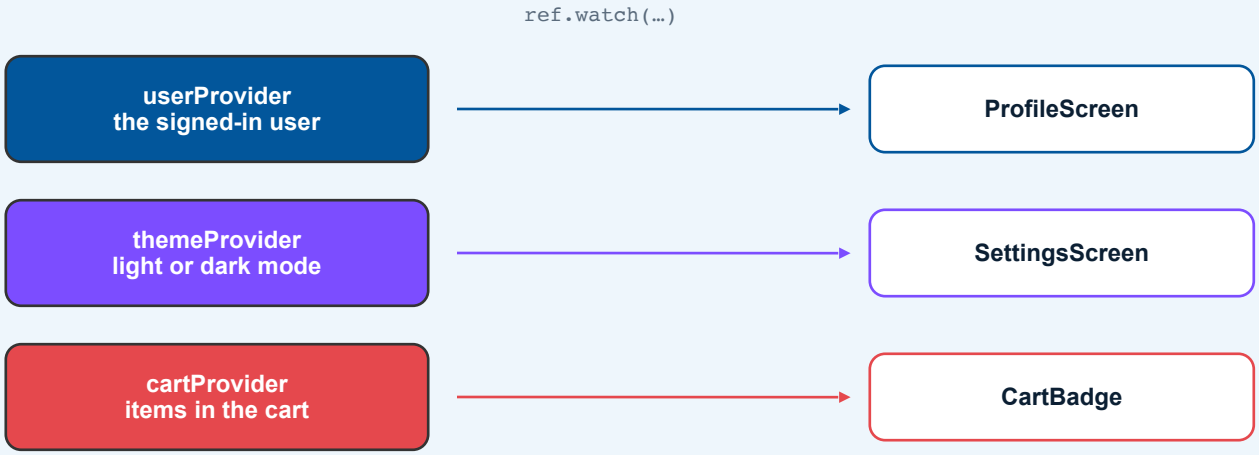
Riverpod revision

Fifteen slides. Everything from Lecture 5 compressed: what a provider is, the six provider types, Ref, AsyncValue, lifecycle, 3.x changes and code generation.

REVISION

What is a provider?

A provider is a named box that holds one piece of state and knows how to create it.



REMEMBER

Any widget can ask for a provider by name.

When the state changes, only the widgets that watch it update themselves.

No passing data down through layers.

- Riverpod creates the provider the first time someone asks, shares one value with everyone, tells widgets to redraw on change, and disposes state nobody uses.

Sources: [3] Getting started, [4] Providers

REVISION

Riverpod in three steps

- Declare: describe the state once, in a provider. A Notifier holds the initial value in build() and the methods that change it.
- Watch: a ConsumerWidget calls ref.watch(provider) inside build(). It gets the current value and is rebuilt whenever it changes.
- Change: a callback calls a method on the notifier with ref.read(provider.notifier). Assigning to state notifies every watcher.
- That is the whole idea. Every other feature is a refinement of these three steps.

counter_app.dart

```dart
// 1. Declare
class Counter extends Notifier<int> {
  @override
  int build() => 0;            // start value
  void increment() => state++; // how it changes
}
final counterProvider =
    NotifierProvider<Counter, int>(Counter.new);

// 2. Watch
class CounterText extends ConsumerWidget {
  const CounterText({super.key});
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final count = ref.watch(counterProvider);
    return Text('$count');
  }
}

// 3. Change
onPressed: () =>
    ref.read(counterProvider.notifier).increment(),
```

Sources: [3] Getting started, [4] Providers, [5] Consumers

The six provider types

P Provider

A value that never changes by itself: config, a repository, an HTTP client, or data computed from other providers.

F FutureProvider

Runs one async function and exposes the result as an AsyncValue. Cached until invalidated.

S StreamProvider

Subscribes to a Stream and exposes the latest event as an AsyncValue.

N NotifierProvider

Synchronous state plus the methods that change it. The default for user-driven state.

A AsyncNotifierProvider

Async state loaded in build() plus methods that modify it. The feature-screen workhorse.

SN StreamNotifierProvider

Stream-backed state with methods. Override updateShouldNotify to tune filtering.

Sources: [4] Providers

REVISION

Provider: values and dependency injection

- Provider exposes a value created once and cached. It only rebuilds when a provider it watches changes.
- Chaining providers with ref.watch is dependency injection: the HTTP client needs the base URL, the repository needs the client.
- ref.onDispose releases resources (close a client, cancel a timer) when the provider is destroyed.
- In tests, override any provider to swap a real dependency for a fake. No service locator, no singletons.

providers.dart

```dart
final apiBaseUrlProvider = Provider<String>((ref) {
  return 'https://api.example.com';
});

final httpClientProvider = Provider<Dio>((ref) {
  final dio = Dio(BaseOptions(
    baseUrl: ref.watch(apiBaseUrlProvider),
  ));
  ref.onDispose(dio.close); // release resources
  return dio;
});

final todoRepositoryProvider =
    Provider<TodoRepository>((ref) {
  return HttpTodoRepository(
    ref.watch(httpClientProvider),
  );
});

// Test: swap the real client for a fake
ProviderScope(overrides: [
  httpClientProvider.overrideWithValue(FakeDio()),
], child: const MyApp());
```

Sources: [4] Providers, [6] Refs

REVISION

Ref: watch, read, listen

W ref.watch

Reads and subscribes. The caller (widget or provider) rebuilds when the value changes. Use inside build().

R ref.read

Reads once without subscribing. Never rebuilds. Use inside onPressed, initState and notifier methods.

L ref.listen

Runs a callback with (previous, next) on every change. No rebuild. Use for snackbars, dialogs, navigation.

ref_usage.dart

```dart
class ProfilePage extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // watch: rebuild when the user changes
    final user = ref.watch(userProvider);

    // listen: side effect, no rebuild
    ref.listen(authProvider, (prev, next) {
      if (next == null) context.go('/login');
    });

    return Column(children: [
      Text(user.name),
      ElevatedButton(
        // read: one-off call in a callback
        onPressed: () => ref
            .read(userProvider.notifier)
            .refresh(),
        child: const Text('Refresh'),
      ),
    ]);
  }
}
```

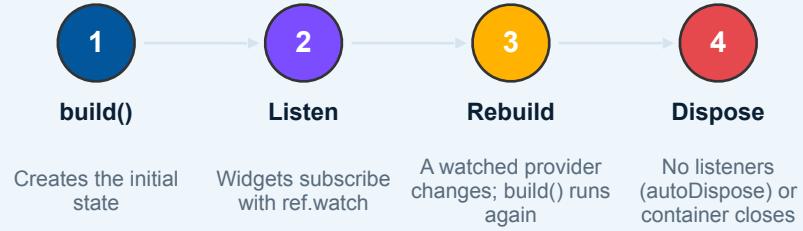
Sources: [6] Refs, [18] DO/DON'T

WidgetRef do and don't

Do	Don't
• ref.watch in build() for every value you display • ref.read inside onPressed and other callbacks • ref.listen for snackbars, dialogs and navigation • select() to rebuild only when one field changes • Keep state immutable; replace it, never mutate it • Guard code after an await with ref.mounted (3.x)	• ref.read in build() for a value on screen: it never updates • ref.watch inside a callback: creates a subscription at the wrong time • Side effects inside build(): build runs many times • Mutating a list inside state: listeners are not notified • Using ref after the widget or provider was disposed • Long-lived mutable fields on a Notifier (3.x recreates it)

Sources: [5] Consumers, [18] DO/DON'T

NotifierProvider and the Notifier lifecycle



- state is the only mutable field and it is replaced, not edited. Assigning a new value notifies all listeners.
- build() returns the initial state and may ref.watch other providers; if they change, build() runs again.
- Public methods form the API of the feature. Widgets call them with ref.read(provider.notifier).

search_notifier.dart

```dart
class Search extends Notifier<List<Item>> {
  @override
  List<Item> build() {
    // rebuild when the query changes
    final query = ref.watch(queryProvider);
    final items = ref.watch(itemsProvider);
    ref.onDispose(() => debugPrint('cleanup'));
    return items
        .where((i) => i.name.contains(query))
        .toList();
  }

  void clear() => state = const [];
}

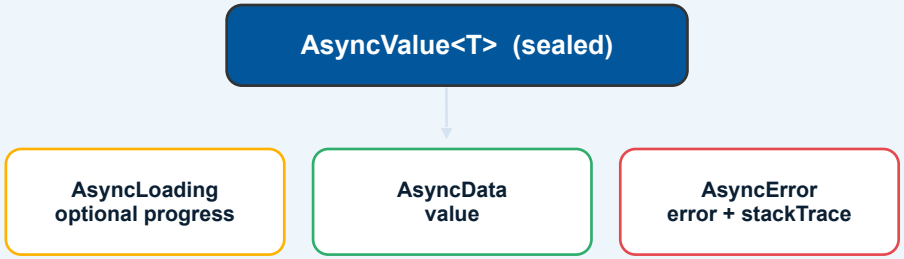
final searchProvider =
    NotifierProvider<Search, List<Item>>(Search.new);

// In 3.0 a new Notifier instance is created on
// every rebuild, so do not keep long-lived
// mutable fields on it.
```

Sources: [4] Providers, [7] Automatic disposal

REVISION

AsyncValue: loading, data, error



READ

value (nullable)
error, stackTrace
requireValue (throws if no value)
hasValue, hasError

FLAGS

isLoading
isRefreshing (forced recompute)
isReloading (dependency changed)
retrying (3.x auto retry)
isFromCache (3.x offline)

async_ui.dart

```dart
final todos = ref.watch(todoListProvider);

// 1. Pattern matching (exhaustive)
return switch (todos) {
  AsyncData(:final value) => TodoListView(value),
  AsyncError(:final error) => ErrorView('$error'),
  AsyncLoading() => const Center(
      child: CircularProgressIndicator()),
};

// 2. Callbacks
todos.when(
  data: (items) => TodoListView(items),
  error: (e, st) => ErrorView('$e'),
  loading: () => const CircularProgressIndicator(),
);

// 3. Reuse the last value while refreshing
final items = todos.value ?? const [];
if (todos.isRefreshing) showLinearProgress();
```

Sources: [14] AsyncValue API reference

REVISION

AsyncNotifier: async state with methods

PURPOSE

Loads data asynchronously in build() and exposes methods that modify it, usually by calling a server. Loading and error states come for free.

KEY PATTERN

Set state = AsyncLoading(), then state = await AsyncValue.guard(…). guard turns any thrown exception into AsyncError instead of crashing the app.

USE IT WHEN

A screen loads remote data and also creates, updates or deletes it. This is the standard shape of a feature controller in Riverpod.

todo_list.dart

```dart
class TodoList extends AsyncNotifier<List<Todo>> {
  @override
  Future<List<Todo>> build() {
    final repo = ref.watch(todoRepositoryProvider);
    return repo.fetchAll();   // initial load
  }

  Future<void> add(String title) async {
    final repo = ref.read(todoRepositoryProvider);
    state = const AsyncLoading();
    state = await AsyncValue.guard(() async {
      await repo.create(title);
      return repo.fetchAll();
    });
  }
}

final todoListProvider =
    AsyncNotifierProvider<TodoList, List<Todo>>(
        TodoList.new);
```

Sources: [4] Providers, [14] AsyncValue API

family, autoDispose and keepAlive



- family: one cached state per argument (userProvider('42') and userProvider('7') are separate). Arguments need == and hashCode.
- autoDispose: free the state when nobody listens. keepAlive() overrides that for as long as you want.
- Rule of thumb: screen data is autoDispose, app-wide data is kept alive. With codegen, autoDispose is the default.

family_dispose.dart

```dart
// One cached state per product id
final productProvider =
    FutureProvider.autoDispose.family<Product, String>(
        (ref, id) {
  // Keep the result 30 s after the screen closes
  final link = ref.keepAlive();
  final timer = Timer(
      const Duration(seconds: 30), link.close);
  ref.onDispose(timer.cancel);
  return ref.watch(productRepositoryProvider).fetch(id);
});

ref.watch(productProvider('42')); // own state
ref.watch(productProvider('7'));  // own state

// 3.x: Notifier family takes the arg in the ctor
class ItemCounter extends Notifier<int> {
  ItemCounter(this.itemId);
  final String itemId;
  @override
  int build() => 0;
}
```

Sources: [8] Family, [7] Automatic disposal

REVISION

select, invalidate and refresh

S **select((state) => slice)**

Watch one field of a provider. The widget rebuilds only when that field changes. selectAsync does the same for async providers.

I **ref.invalidate(provider)**

Marks the state stale. It recomputes when next needed, or immediately if someone is listening. Use on logout to reset state.

R **ref.refresh(provider)**

Invalidates and returns the new value immediately. Await provider.future for async providers; the classic pull-to-refresh.

select_refresh.dart

```dart
// Rebuilds only when the name changes
final name = ref.watch(
  userProvider.select((user) => user.name),
);

// Cart badge: rebuild on count change only
final count = ref.watch(
  cartProvider.select((items) => items.length),
);

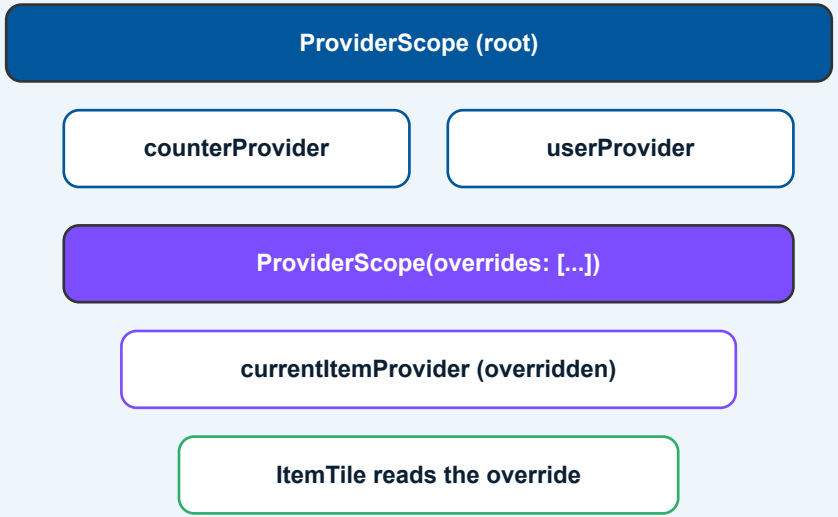
// Pull to refresh
RefreshIndicator(
  onRefresh: () =>
      ref.refresh(todoListProvider.future),
  child: const TodoListView(),
)

// Log out: reset dependent state
ref.invalidate(todoListProvider);
ref.invalidate(userProvider);
```

Sources: [17] Reduce rebuilds, [6] Refs

REVISION

ProviderScope, overrides and testing



- ProviderContainer is the same thing without widgets: use it in pure Dart and in unit tests. Always dispose it.

counter_test.dart

```dart
void main() {
  test('increment adds one', () {
    final container = ProviderContainer();
    addTearDown(container.dispose);

    expect(container.read(counterProvider), 0);
    container.read(counterProvider.notifier)
        .increment();
    expect(container.read(counterProvider), 1);
  });

  test('uses a fake repository', () async {
    final container = ProviderContainer(overrides: [
      todoRepositoryProvider
          .overrideWithValue(FakeTodoRepository()),
    ]);
    addTearDown(container.dispose);
    final todos =
        await container.read(todoListProvider.future);
    expect(todos, isEmpty);
  });
}
```

Sources: [13] Testing, [16] Scoping, [22] Containers

What changed in Riverpod 3.x

U Unified API

No AutoDispose or Family variants. One Notifier class and one Ref; family arguments move into the constructor.

R Automatic retry

Failing providers retry with exponential backoff, 200 ms doubling to 6.4 s. Return null from retry to stop.

P Pause / resume

Listeners in off-screen widgets are paused via TickerMode. A provider used only by paused providers pauses too.

M Ref.mounted

Using a disposed Ref throws UnmountedRefException. Guard code after await with if (!ref.mounted) return.

= Filtering with ==

All providers skip updates whose new state == the old one. Give state classes value equality.

E Mutations, offline

Experimental: Mutation<T> models side-effect lifecycle; persist() caches state in SQLite with riverpod_sqflite.

Sources: [1] What's new in 3.0, [2] Migration guide

Code generation with @riverpod

- Annotate a function or class with @riverpod; the generator writes the provider for you in a .g.dart file.
- Function parameters become family arguments. Named parameters stay named at the call site.
- Generated providers are autoDispose by default. Use @Riverpod(keepAlive: true) to opt out.
- Add riverpod_lint and run dart analyze in CI: it checks ref usage and missing scope overrides.
- Run: dart run build_runner watch -d

todos.dart

```dart
part 'todos.g.dart';

// Functional provider: userProvider('42')
@riverpod
Future<User> user(Ref ref, String id) async {
  return ref.watch(userRepositoryProvider).fetch(id);
}

// Lives for the whole app session
@Riverpod(keepAlive: true)
AuthRepository authRepository(Ref ref) =>
    AuthRepository(ref.watch(httpClientProvider));

// Class provider: todoListProvider
@riverpod
class TodoList extends _$TodoList {
  @override
  Future<List<Todo>> build() =>
      ref.watch(todoRepositoryProvider).fetchAll();

  Future<void> add(String title) async { /* ... */ }
}
```

Sources: [12] About code generation

Riverpod: what to keep in mind

1 Declarative dependency graph

Providers describe what state is and what it depends on. Riverpod decides when to build, share and rebuild.

2 Notifiers own mutations

State changes only through notifier methods. Business logic lives outside widgets and is unit-testable.

3 AsyncValue is exhaustive

The sealed type forces you to handle loading, data and error. switch gives compile-time safety.

4 Override everything

ProviderScope and ProviderContainer overrides swap any dependency for tests and nested scopes.

5 Lifecycle is automatic

autoDispose frees unused state; keepAlive and family give you control. 3.x adds retry and pause.

6 Codegen is optional

@riverpod removes boilerplate and can be mixed with hand-written providers.

Sources: [3] Getting started

PART 2

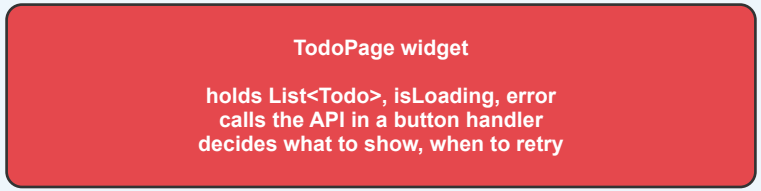
Bloc and Cubit

Event-driven state management with flutter_bloc: Cubit for simple state, Bloc for event-driven state, stream-based transformations, functional state isolation and clean architecture layers.

START HERE

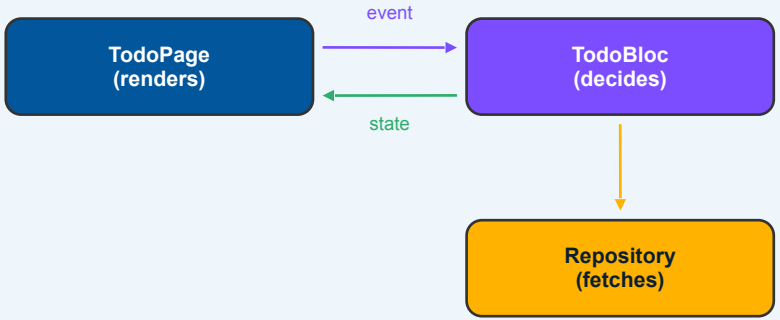
The problem: logic tangled inside widgets

WITH setState



Hard to test (needs a widget tree). Hard to reuse (another screen needs the same logic). Hard to reason about (UI and business rules interleaved).

WITH BLOC



The widget only sends events and renders states. The Bloc holds all the logic and is a plain Dart class you can test with no Flutter at all.

Bloc —> Business Logic Component. **Bloc separates "what happened" (events) from "what to show" (states).**

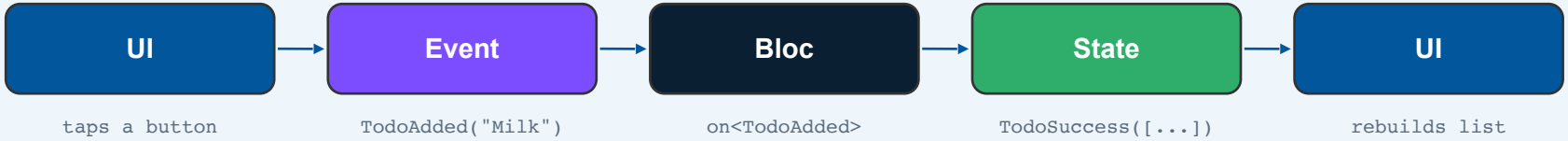
UI is a pure function of state: given the same state, you always get the same screen.

Sources: [B1] bloclibrary.dev, Core concepts

START HERE

The BLoC pattern: Business Logic Component

A Bloc takes a Stream of events in and produces a Stream of states out.



E Event

Something that happened: a tap, a page load, a timer tick. A plain Dart object, usually a sealed class.

S State

A snapshot of what the UI should show right now. Immutable. Replaced, never mutated.

B Bloc

Maps each event to zero or more new states. Holds business logic. Exposes a Stream<State>.

C Cubit

A Bloc without events: methods call emit(newState) directly. Less boilerplate.

W Widgets

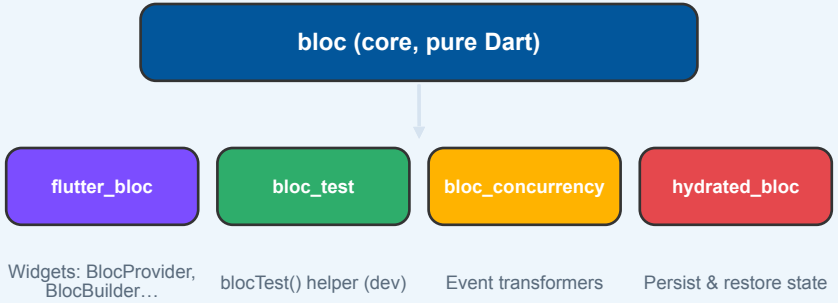
BlocProvider creates the bloc; BlocBuilder rebuilds on state; BlocListener runs side effects.

R Repository

Fetches and stores data. Injected into the bloc so it can be faked in tests.

Sources: [B1] bloclibrary.dev, Core concepts

The bloc package family and setup



- bloc is pure Dart: you can use it in CLI tools and servers. flutter_bloc adds the widgets.
- bloc_test and bloc_concurrency are small add-ons you will use in almost every app.
- Setting Bloc.observer gives you a global log of every event, transition and error.

terminal + main.dart

```
# Terminal (latest: flutter_bloc ^9.x)
flutter pub add flutter_bloc
flutter pub add bloc_concurrency
flutter pub add dev:bloc_test
flutter pub add dev:mocktail

// main.dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

void main() {
  // Optional: log every transition
  Bloc.observer = AppBlocObserver();

  runApp(
    RepositoryProvider(
      create: (_) => TodoRepository(),
      child: const MyApp(),
    ),
  );
}
```

Sources: [B2] pub.dev/packages/flutter_bloc

Streams: the engine under Bloc

WHAT IS A STREAM

A sequence of values delivered over time. A Future gives you one value later; a Stream gives you many values, one by one, until it closes.

THREE WAYS TO PRODUCE ONE

async* with yield (a generator function)
StreamController and its sink.add(value)
Transforming another stream with map, where, etc.

WHY IT MATTERS FOR BLOC

Every Bloc wraps a StreamController. emit(state) pushes to the stream; BlocBuilder listens to it; close() closes it. Understanding streams means understanding Bloc.

streams_primer.dart

```dart
// 1. Generator: produce values over time
Stream<int> countTo(int n) async* {
  for (var i = 1; i <= n; i++) {
    await Future.delayed(const Duration(seconds: 1));
    yield i;                // push one value
  }
}
countTo(3).listen(print);  // 1, 2, 3

// 2. StreamController: push manually
final controller = StreamController<String>();
controller.stream.listen((msg) => print(msg));
controller.sink.add('hello');
controller.sink.add('world');
await controller.close(); // always close!

// 3. Transform an existing stream
final doubled = countTo(3).map((n) => n * 2);
final evens = countTo(10).where((n) => n.isEven);

// Consume with await for
await for (final n in countTo(3)) { print(n); }
```

Sources: [B3] dart.dev/libraries/async/using-streams

Stream operators you will meet in Bloc

map

Transform each value

ints → strings

where

Keep only matching values

filter

distinct

Skip repeated consecutive values

dedupe

debounce

Wait for a pause before emitting

search box

throttle

At most one value per window

scroll events

switchMap

Cancel the previous inner stream

latest only

operators.dart

```dart
import 'package:stream_transform/stream_transform.dart';

final clicks = StreamController<int>();

// map + where: only even numbers, as labels
clicks.stream
    .where((n) => n.isEven)
    .map((n) => 'Click #$n')
    .listen(print);

// debounce: emit after 300 ms of silence
textChanges
    .debounce(const Duration(milliseconds: 300))
    .listen(search);

// switchMap: new query cancels the old request
textChanges
    .switchMap((q) => api.search(q).asStream())
    .listen(showResults);

// distinct: ignore 'a','a','a' -> one 'a'
textChanges.distinct().listen(print);
```

Sources: [B4] pub.dev/packages/stream_transform

PART 3

Cubit

The simplest way to use the bloc library: a class with a state and methods that emit new states. Start here, upgrade to Bloc when you need events.

What is a Cubit?

A Cubit is a Bloc without events: you call methods, the methods call emit().



- Cubit<State> holds the current state and a stream of states. The constructor passes the initial state to super.
- Each public method is an intention: increment(), load(), toggle(). Inside, emit(newState) notifies listeners.
- Compare with a Riverpod Notifier: the same shape, but emit(x) instead of state = x, and widgets use BlocBuilder instead of ref.watch.

counter_cubit.dart

```dart
import 'package:flutter_bloc/flutter_bloc.dart';

class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);   // initial state

  void increment() => emit(state + 1);
  void decrement() => emit(state - 1);
  void reset()     => emit(0);
}

// Pure Dart usage (no widgets)
final cubit = CounterCubit();
cubit.stream.listen(print);  // 1, 2, 0
cubit.increment(); cubit.increment(); cubit.reset();
await cubit.close();
```

Sources: [B1] bloclibrary.dev, Cubit

Cubit in three steps: provide, build, call

- Provide: BlocProvider creates the cubit above the widgets that need it and closes it automatically when removed from the tree.
- Build: BlocBuilder<CounterCubit, int> rebuilds its builder every time the cubit emits a new state.
- Call: context.read<CounterCubit>() returns the cubit without subscribing. Use it in callbacks, exactly like ref.read.
- Never call context.read inside build() for a displayed value; use BlocBuilder or context.watch instead.

counter_page.dart

```dart
// 1. Provide
BlocProvider(
  create: (_) => CounterCubit(),
  child: const CounterPage(),
)

class CounterPage extends StatelessWidget {
  const CounterPage({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // 2. Build: rebuilds on every new int
      body: BlocBuilder<CounterCubit, int>(
        builder: (context, count) =>
            Center(child: Text('$count')),
      ),
      floatingActionButton: FloatingActionButton(
        // 3. Call: read, do not subscribe
        onPressed: () =>
            context.read<CounterCubit>().increment(),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Sources: [B5] bloclibrary.dev, Flutter counter tutorial

Rich state: a class instead of a primitive

WHY A STATE CLASS

Real screens need several values at once: the data, whether it is loading, and an error message. A single immutable class keeps them consistent.

copyWith

Returns a new object with only some fields changed. This is how you "update" immutable state without touching the old object.

EQUALITY

Bloc skips emit() when newState == state. Override == and hashCode (or use equatable / freezed) so that equal states are recognised.

todo_state.dart

```dart
enum TodoStatus { initial, loading, success, failure }

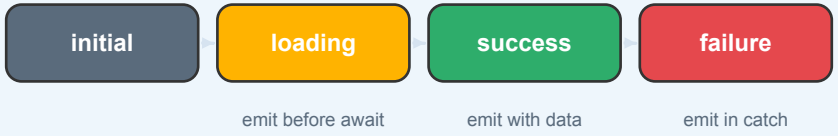
@immutable
class TodoState {
  const TodoState({
    this.todos = const [],
    this.status = TodoStatus.initial,
    this.error,
  });

  final List<Todo> todos;
  final TodoStatus status;
  final String? error;

  TodoState copyWith({
    List<Todo>? todos,
    TodoStatus? status,
    String? error,
  }) {
    return TodoState(
      todos: todos ?? this.todos,
      status: status ?? this.status,
      error: error,   // null resets the error
    );
  }
}
```

Sources: [B1] bloclibrary.dev, Core concepts

Async loading in a Cubit



- The pattern is always the same: emit loading, await the repository, emit success or failure.
- The repository is injected through the constructor. In tests you pass a fake one.
- After an await the cubit may already be closed (user left the page). Check isClosed before emitting.
- This mirrors AsyncValue.guard in Riverpod, written out by hand.

todo_cubit.dart

```dart
class TodoCubit extends Cubit<TodoState> {
  TodoCubit(this._repository)
      : super(const TodoState());

  final TodoRepository _repository;

  Future<void> load() async {
    emit(state.copyWith(status: TodoStatus.loading));
    try {
      final todos = await _repository.fetchAll();
      if (isClosed) return;       // page was left
      emit(state.copyWith(
        todos: todos,
        status: TodoStatus.success,
      ));
    } catch (e) {
      emit(state.copyWith(
        status: TodoStatus.failure,
        error: e.toString(),
      ));
    }
  }

  void remove(String id) => emit(state.copyWith(
      todos: state.todos.where((t) => t.id != id).toList()));
}
```

Sources: [B1] bloclibrary.dev, Cubit

Cubit or Bloc? How to choose

Cubit	Bloc
• Methods on the class; call emit() directly • Less code: no event classes to write • Simple, linear transitions (counter, toggle, form field, theme) • Hard to trace later why a state changed • No way to control how fast calls are processed	• Events are objects; a handler per event type • More code, but every change has a name • BlocObserver logs event → state transitions • Event transformers: debounce, drop, restart • Better for complex flows: search, auth, sync

Start with Cubit. Switch to Bloc when you need traceability or event transformers.

Both share the same widgets (BlocProvider, BlocBuilder), so switching later is cheap.

Sources: [B1] bloclibrary.dev, Cubit vs Bloc

PART 4

Bloc and events

Sealed events and states, registering handlers with on<Event>, the Emitter, a complete TodoBloc, and event transformers that control concurrency.

Events: naming what happened

WHAT

An event is a plain Dart object describing something that happened, named in the past tense: TodoAdded, PageRefreshed, LogoutRequested.

WHY SEALED

A sealed base class means the compiler knows every subtype. A switch over events must handle them all, and adding a new event flags every place that forgot it.

DATA ON THE EVENT

Carry only what the handler needs: the title to add, the id to delete. Keep events small and immutable (const constructors, final fields).

todo_event.dart

```dart
// Base class: sealed so switches are exhaustive
sealed class TodoEvent {
  const TodoEvent();
}

// Page opened or pull-to-refresh
final class TodoLoadRequested extends TodoEvent {
  const TodoLoadRequested();
}

// User submitted the "new todo" field
final class TodoAdded extends TodoEvent {
  const TodoAdded(this.title);
  final String title;
}

// User swiped a row away
final class TodoRemoved extends TodoEvent {
  const TodoRemoved(this.id);
  final String id;
}

// User tapped the checkbox
final class TodoToggled extends TodoEvent {
  const TodoToggled(this.id);
  final String id;
}
```

Sources: [B1] bloclibrary.dev, Bloc

States: what the screen should show

TWO STYLES

Sealed subclasses (Loading, Success, Failure) when each state carries different data. A single class with a status enum when fields are shared (see slide 28).

IMMUTABLE

A state is never edited. Every change creates a new object with emit(). The old object can safely be compared with the new one.

RENDER WITH switch

switch (state) { TodoSuccess(:final todos) => …, TodoFailure(:final error) => …, _ => spinner } is exhaustive and reads like a specification of the UI.

todo_state.dart

```dart
sealed class TodoState {
  const TodoState();
}

final class TodoInitial extends TodoState {
  const TodoInitial();
}

final class TodoLoading extends TodoState {
  const TodoLoading();
}

final class TodoSuccess extends TodoState {
  const TodoSuccess(this.todos);
  final List<Todo> todos;
}

final class TodoFailure extends TodoState {
  const TodoFailure(this.message);
  final String message;
}

// In the widget
Widget build(BuildContext context, TodoState state) =>
  switch (state) {
    TodoSuccess(:final todos) => TodoListView(todos),
    TodoFailure(:final message) => ErrorView(message),
    _ => const CircularProgressIndicator(),
  };
```

Sources: [B1] bloclibrary.dev, Bloc

The Bloc class: on<Event> and Emitter

- Bloc<Event, State>: the constructor passes the initial state to super and registers one handler per event type with on<E>().
- A handler receives the event and an Emitter. Call emit(state) as many times as you like, including after awaits.
- The Emitter is only valid while the handler runs. Do not store it or emit after the handler returned.
- Widgets send events with context.read<TodoBloc>().add(const TodoAdded('Milk')).
- The constructor runs once; handlers run every time a matching event arrives.

todo_bloc.dart

```dart
class TodoBloc extends Bloc<TodoEvent, TodoState> {
  TodoBloc(this._repository) : super(const TodoInitial()) {
    // Register a handler per event type
    on<TodoLoadRequested>(_onLoadRequested);
    on<TodoAdded>(_onAdded);
    on<TodoRemoved>(_onRemoved);
  }

  final TodoRepository _repository;

  Future<void> _onLoadRequested(
    TodoLoadRequested event,
    Emitter<TodoState> emit,
  ) async {
    emit(const TodoLoading());
    try {
      final todos = await _repository.fetchAll();
      emit(TodoSuccess(todos));
    } catch (e) {
      emit(TodoFailure(e.toString()));
    }
  }
  // _onAdded, _onRemoved on the next slide
}
```

Sources: [B1] bloclibrary.dev, Bloc

Complete TodoBloc: add, remove, toggle

- Each handler reads the current state, computes a new one, and emits it.
- if (state is! TodoSuccess) return; guards against events that arrive while loading or after a failure.
- Lists are copied with the spread operator or collection-for; the old list is never mutated.
- Writes go through the repository first; if the call throws, the catch block emits TodoFailure and the UI can show a retry.
- A handler can also re-dispatch: add(const TodoLoadRequested()) reloads from the server after a write.

todo_bloc.dart (continued)

```dart
Future<void> _onAdded(
    TodoAdded event, Emitter<TodoState> emit) async {
  final current = state;
  if (current is! TodoSuccess) return;
  try {
    final created = await _repository.create(event.title);
    emit(TodoSuccess([...current.todos, created]));
  } catch (e) {
    emit(TodoFailure('Could not add: $e'));
  }
}

Future<void> _onRemoved(
    TodoRemoved event, Emitter<TodoState> emit) async {
  final current = state;
  if (current is! TodoSuccess) return;
  await _repository.delete(event.id);
  emit(TodoSuccess([
    for (final t in current.todos)
      if (t.id != event.id) t,
  ]));
}

// In a widget:
context.read<TodoBloc>().add(const TodoAdded('Milk'));
```

Sources: [B1] bloclibrary.dev, Bloc

Event transformers: what if events arrive too fast?

A transformer decides how overlapping events of the same type are processed.

The user types "flu", "flut", "flutt" in 200 ms; each is a SearchQueryChanged event. Which requests run?

`concurrent()`

Default. Every event runs in parallel. Responses can arrive out of order.

Use for: fire-and-forget actions

`sequential()`

Queue. Each handler waits for the previous one to finish. Order is safe.

Use for: ordered writes

`droppable()`

Ignore new events while one is running. Prevents double submits.

Use for: submit buttons

`restartable()`

Cancel the running handler and start the newest. Latest wins.

Use for: search as you type

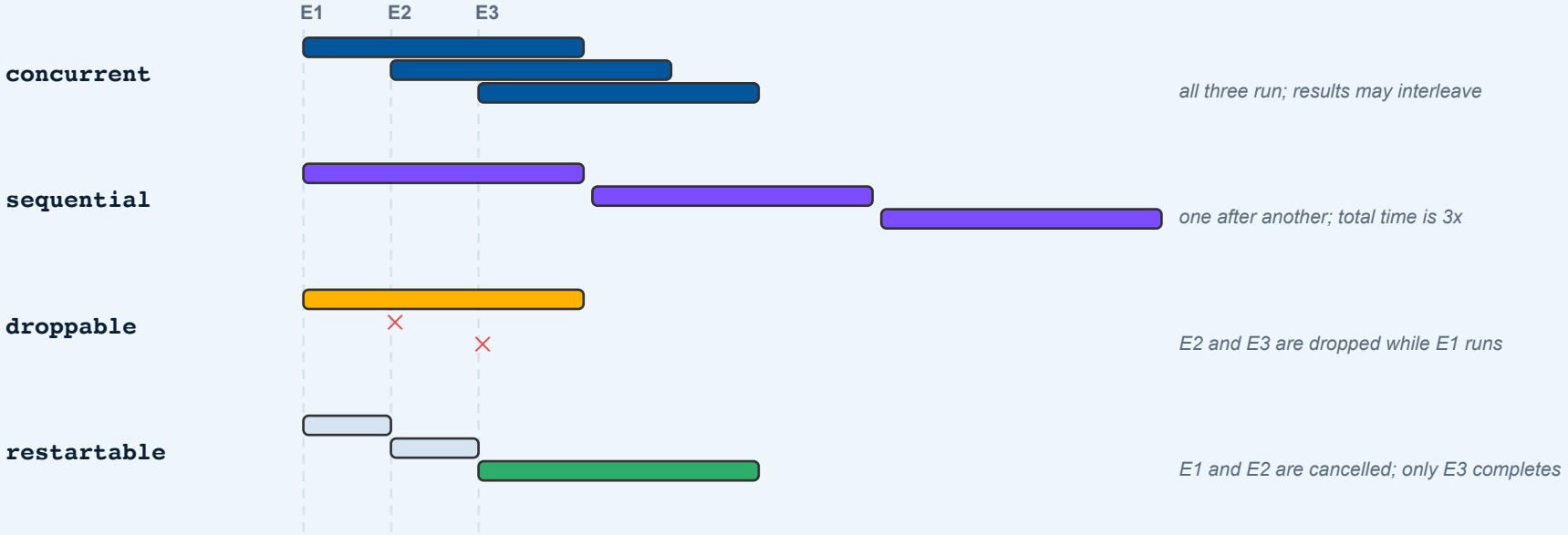
search_bloc.dart

```dart
import 'package:bloc_concurrency/bloc_concurrency.dart';   // flutter pub add bloc_concurrency
on<SearchQueryChanged>(_onQueryChanged, transformer: restartable());
```

Sources: [B6] pub.dev/packages/bloc_concurrency

Transformers on a timeline

Three events arrive while handlers are still running. Bars show which handlers execute to completion.



RULE OF THUMB

Search box: restartable. Submit button: droppable. Ordered writes (reorder list, checkout steps): sequential. Independent reads: concurrent.

Sources: [B6] pub.dev/packages/bloc_concurrency

PART 5

flutter_bloc widgets

How the widget tree talks to blocs: BlocProvider, BlocBuilder, BlocListener, BlocConsumer, BlocSelector, context.read and context.watch.

BlocProvider: create, share, dispose

PURPOSE

Creates a bloc and makes it available to every widget below it in the tree via context. When the provider is removed, the bloc is closed automatically.

LAZY BY DEFAULT

create runs the first time a widget asks for the bloc, not when the provider is built. Pass lazy: false to create it immediately (e.g. to start loading).

BlocProvider.value

Passes an existing bloc to a new route or subtree without creating or closing it. Use it when pushing a page that needs the same bloc as its parent.

bloc_provider.dart

```dart
// Create and own the bloc
BlocProvider(
  create: (context) => TodoBloc(
    context.read<TodoRepository>(),
  )..add(const TodoLoadRequested()), // kick off
  child: const TodoPage(),
)

// Several blocs at once
MultiBlocProvider(
  providers: [
    BlocProvider(create: (_) => ThemeCubit()),
    BlocProvider(create: (_) => AuthBloc()),
  ],
  child: const MyApp(),
)

// Reuse an existing bloc on a new route
Navigator.push(context, MaterialPageRoute(
  builder: (_) => BlocProvider.value(
    value: context.read<TodoBloc>(),
    child: const TodoDetailPage(),
  ),
));
```

Sources: [B7] bloclibrary.dev, flutter_bloc concepts

BlocBuilder: rebuild the UI on every state

PURPOSE

Subscribes to the bloc's stream and calls builder(context, state) each time a new state is emitted. The Riverpod equivalent of ref.watch.

buildWhen

Optional filter: return false to skip a rebuild. Useful to ignore transient states, or to rebuild only when a specific field changed.

KEEP BUILDER PURE

Only return widgets. No navigation, no snackbars, no bloc.add() inside builder; those belong in BlocListener. builder may run many times.

todo_list_view.dart

```dart
BlocBuilder<TodoBloc, TodoState>(
  // Skip rebuild for the brief loading flash
  buildWhen: (previous, current) =>
      current is! TodoLoading,
  builder: (context, state) {
    return switch (state) {
      TodoSuccess(:final todos) => ListView.builder(
          itemCount: todos.length,
          itemBuilder: (_, i) => CheckboxListTile(
            title: Text(todos[i].title),
            value: todos[i].done,
            onChanged: (_) => context
                .read<TodoBloc>()
                .add(TodoToggled(todos[i].id)),
          ),
        ),
      TodoFailure(:final message) => Center(
          child: Text('Error: $message')),
      _ => const Center(
          child: CircularProgressIndicator()),
    };
  },
)
```

Sources: [B7] bloclibrary.dev, flutter_bloc concepts

BlocListener: side effects without rebuilding

PURPOSE

Runs listener(context, state) once per state change and does not rebuild its child. The Riverpod equivalent of ref.listen.

USE IT FOR

Showing a snackbar or dialog, navigating to another page, logging analytics, triggering an event on another bloc. Anything that must happen exactly once.

listenWhen

Same idea as buildWhen: only call the listener when the condition holds, e.g. when the state became a failure.

todo_page.dart

```dart
BlocListener<TodoBloc, TodoState>(
  listenWhen: (previous, current) =>
      current is TodoFailure,
  listener: (context, state) {
    if (state is TodoFailure) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(state.message)),
      );
    }
  },
  child: const TodoListView(),   // not rebuilt
)

// Several listeners without nesting
MultiBlocListener(
  listeners: [
    BlocListener<AuthBloc, AuthState>(
      listener: (context, state) {
        if (state is Unauthenticated) {
          context.go('/login');
        }
      },
    ),
    BlocListener<TodoBloc, TodoState>(/* ... */),
  ],
  child: const HomePage(),
)
```

Sources: [B7] bloclibrary.dev, flutter_bloc concepts

BlocConsumer and BlocSelector

C BlocConsumer

BlocBuilder and BlocListener in one widget. Use when the same state drives both the UI and a side effect, so you do not nest two widgets.

S BlocSelector

Picks one value out of the state and rebuilds only when that value changes. The Bloc equivalent of Riverpod select().

ctx context.read / context.watch

read: get the bloc, no subscription (callbacks). watch: get the bloc and rebuild on change (inside build). select: watch one field.

consumer_selector.dart

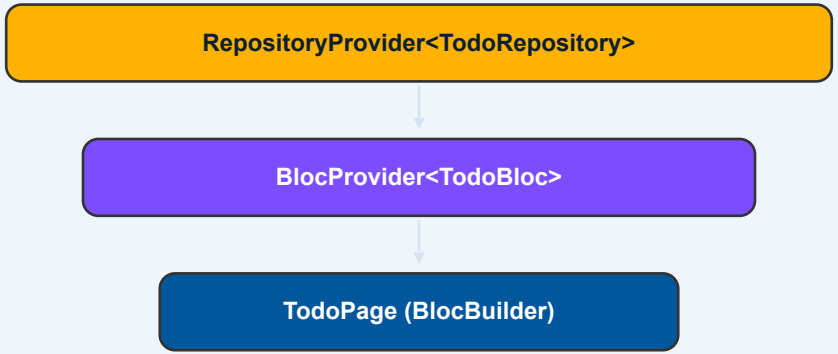
```dart
BlocConsumer<CheckoutBloc, CheckoutState>(
  listener: (context, state) {
    if (state is CheckoutSuccess) {
      context.go('/receipt/${state.orderId}');
    }
  },
  builder: (context, state) => state is CheckoutLoading
      ? const CircularProgressIndicator()
      : const CheckoutForm(),
)

// Rebuild the badge only when the count changes
BlocSelector<CartCubit, CartState, int>(
  selector: (state) => state.items.length,
  builder: (context, count) => Badge(
    label: Text('$count'),
    child: const Icon(Icons.shopping_cart),
  ),
)

// Same thing with extension methods
final count = context.select(
    (CartCubit c) => c.state.items.length);
final cubit = context.read<CartCubit>(); // callbacks
```

Sources: [B7] bloclibrary.dev, flutter_bloc concepts

RepositoryProvider: injecting dependencies



The bloc reads the repository from context in create. The page reads the bloc. Nothing is a global singleton; everything can be replaced.

- RepositoryProvider is for plain classes (repositories, API clients, services); BlocProvider is for blocs and cubits.
- Wrap once at the root with MultiRepositoryProvider; read anywhere with context.read<T>().

app.dart

```dart
void main() {
  runApp(
    MultiRepositoryProvider(
      providers: [
        RepositoryProvider<ApiClient>(
          create: (_) => ApiClient(baseUrl: kApiUrl),
        ),
        RepositoryProvider<TodoRepository>(
          create: (context) => HttpTodoRepository(
            context.read<ApiClient>(),
          ),
        ),
      ],
      child: const MyApp(),
    ),
  );
}

// Later, in a page route
BlocProvider(
  create: (context) => TodoBloc(
    context.read<TodoRepository>(),
  )..add(const TodoLoadRequested()),
  child: const TodoPage(),
)
```

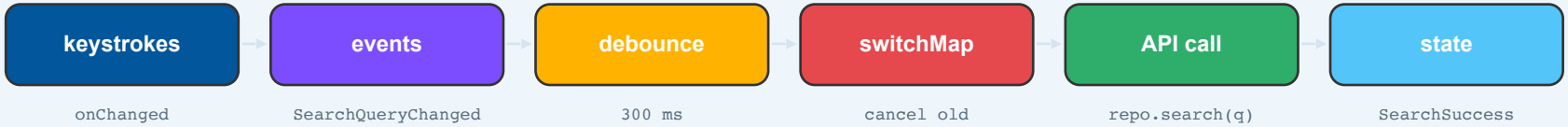
Sources: [B7] bloclibrary.dev, flutter_bloc concepts

PART 6

Stream-based state transformations

Custom event transformers, consuming repository streams with emit.forEach, optimistic updates and the search pipeline from keystroke to results.

Search as you type: the full pipeline



- A transformer is just a function: it takes the stream of events and a mapper that runs the handler, and returns the stream Bloc should process.
- debounce waits until the user pauses; switchMap cancels the handler for an older query when a newer one arrives.
- The handler itself stays simple: emit loading, await, emit results.
- Without this, typing "flutter" fires seven requests and the slowest one may paint last.

search_bloc.dart

```dart
import 'package:stream_transform/stream_transform.dart';

EventTransformer<E> debounceRestartable<E>(
    Duration d) {
  return (events, mapper) =>
      events.debounce(d).switchMap(mapper);
}

class SearchBloc extends Bloc<SearchEvent, SearchState> {
  SearchBloc(this._repo) : super(const SearchInitial()) {
    on<SearchQueryChanged>(
      _onQueryChanged,
      transformer: debounceRestartable(
          const Duration(milliseconds: 300)),
    );
  }
  final SearchRepository _repo;

  Future<void> _onQueryChanged(SearchQueryChanged e,
      Emitter<SearchState> emit) async {
    if (e.query.isEmpty) return emit(const SearchInitial());
    emit(const SearchLoading());
    emit(SearchSuccess(await _repo.search(e.query)));
  }
}
```

Sources: [B6] bloc_concurrency, [B4] stream_transform

emit.forEach: repository stream to states

THE PROBLEM

Firestore, a WebSocket or a local database gives you a Stream of data. You want every event to become a new state without managing a StreamSubscription by hand.

emit.forEach

Subscribes to the stream, maps each item to a state with onData, and keeps the handler alive until the stream closes or the bloc is closed. Cleanup is automatic.

emit.onEach

Same idea, but you call emit yourself inside the callback. Use it when one stream item should produce several states or trigger other work.

messages_bloc.dart

```dart
class MessagesBloc extends Bloc<MessagesEvent, MessagesState> {
  MessagesBloc(this._repo) : super(const MessagesInitial()) {
    // restartable: switching chat cancels old stream
    on<ChatOpened>(_onChatOpened,
        transformer: restartable());
  }
  final ChatRepository _repo;

  Future<void> _onChatOpened(
      ChatOpened event, Emitter<MessagesState> emit) {
    emit(const MessagesLoading());
    // Each stream event becomes a new state
    return emit.forEach<List<Message>>(
      _repo.watchMessages(event.chatId),
      onData: (messages) => MessagesSuccess(messages),
      onError: (error, _) =>
          MessagesFailure(error.toString()),
    );
  }
}

// The stream is cancelled when the bloc closes
// or when a newer ChatOpened event restarts it.
```

Sources: [B1] bloclibrary.dev, Emitter

Optimistic updates: show first, confirm later

1. Emit the new state immediately
2. Call the server
3a. Success: keep it
3b. Failure: emit the old state back

- The user sees the checkbox flip instantly; a slow network does not block the UI.
- Keep a reference to the previous state so you can roll back.
- Use sequential() so two toggles on the same item cannot race each other.

todo_bloc.dart (toggle)

```dart
on<TodoToggled>(_onToggled, transformer: sequential());

Future<void> _onToggled(
    TodoToggled event, Emitter<TodoState> emit) async {
  final previous = state;
  if (previous is! TodoSuccess) return;

  // 1. Optimistic: flip it on screen right away
  emit(TodoSuccess([
    for (final t in previous.todos)
      if (t.id == event.id) t.copyWith(done: !t.done)
      else t,
  ]));

  try {
    // 2. Confirm with the server
    await _repository.toggle(event.id);
  } catch (e) {
    // 3b. Roll back and tell the user
    emit(previous);
    emit(TodoFailure('Could not update: $e'));
    emit(previous); // return to the list
  }
}
```

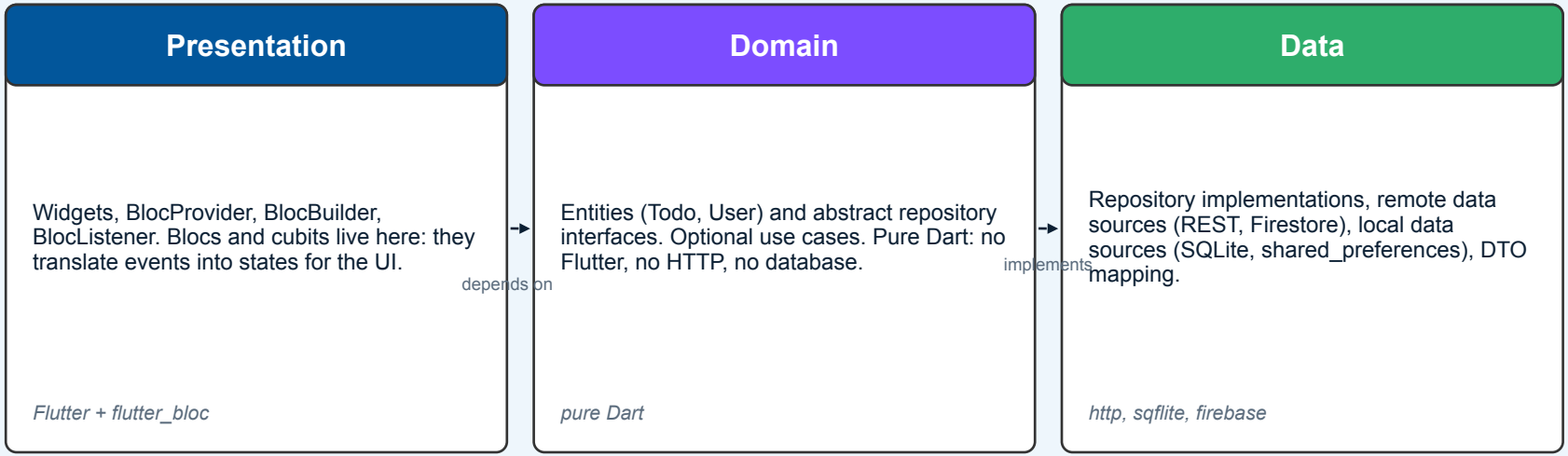
Sources: [B1] bloclibrary.dev, Bloc

PART 7

Clean architecture layers

Presentation, domain and data. The dependency rule, the repository pattern, functional state isolation between blocs, and BlocObserver for cross-cutting concerns.

Three layers and the dependency rule



THE DEPENDENCY RULE

Source code dependencies point inward. Presentation knows Domain; Data knows Domain; Domain knows nothing about the other two. Swap the database or the UI without touching business rules.

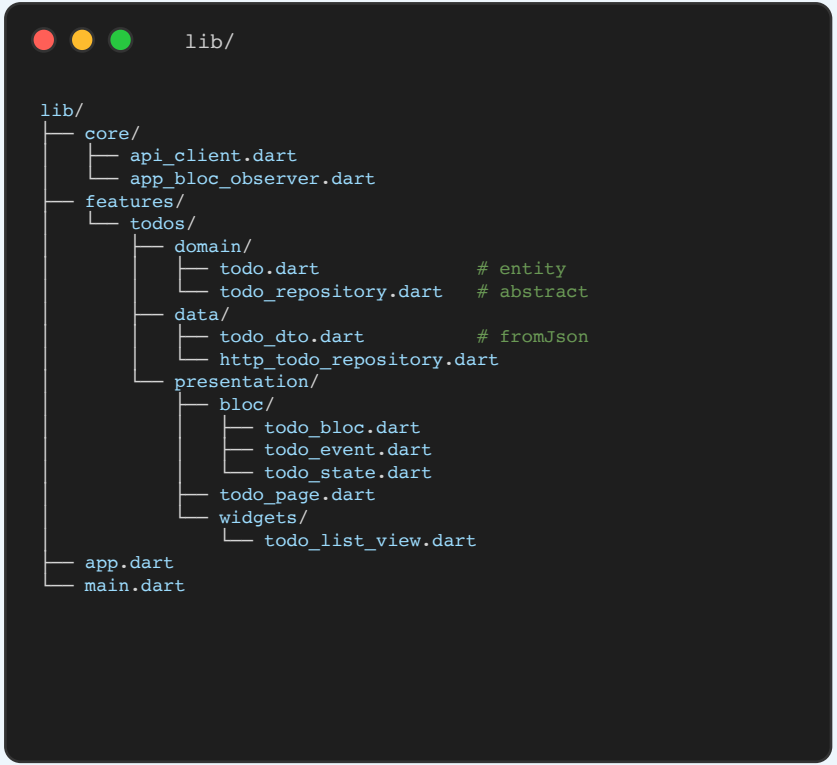
WHERE DO BLOCS GO

Blocs are presentation logic: they turn user intent into screen state. They depend on domain interfaces, never on concrete data classes. That is what makes them testable.

Sources: [B8] Clean Architecture, R. C. Martin; [B9] Flutter architecture guide

A feature-first folder structure

- Group by feature (todos, auth, cart), then by layer inside each feature. A feature folder can be deleted or moved as a unit.
- presentation holds the bloc and the widgets; domain holds entities and the repository interface; data holds the implementation and DTOs.
- Shared code (ApiClient, theme, routing) lives in core/.
- Tests mirror the structure: test/features/todos/presentation/ todo_bloc_test.dart.
- Import rule: presentation imports domain; data imports domain; domain imports nothing from the feature.



Sources: [B9] Flutter architecture guide

The repository pattern

INTERFACE IN DOMAIN

An abstract class that says what you can do with todos (fetch, create, delete) in terms of entities. No JSON, no HTTP types.

IMPLEMENTATION IN DATA

Talks to the network or database, maps DTOs to entities, handles retries and caching. There can be several: Http, Local, Fake.

WHY

The bloc depends only on the interface. You can test it with a fake in memory, and later replace REST with GraphQL without changing a single bloc.

todo_repository.dart

```dart
// domain/todo_repository.dart
abstract class TodoRepository {
  Future<List<Todo>> fetchAll();
  Future<Todo> create(String title);
  Future<void> delete(String id);
  Stream<List<Todo>> watchAll();
}

// data/http_todo_repository.dart
class HttpTodoRepository implements TodoRepository {
  HttpTodoRepository(this._client);
  final ApiClient _client;

  @override
  Future<List<Todo>> fetchAll() async {
    final json = await _client.getList('/todos');
    return json.map(TodoDto.fromJson)
               .map((dto) => dto.toEntity())
               .toList();
  }

  @override
  Future<Todo> create(String title) async {
    final json = await _client.post('/todos',
        body: {'title': title});
    return TodoDto.fromJson(json).toEntity();
  }
  // delete, watchAll ...
}
```

Sources: [B8] Clean Architecture; [B9] Flutter architecture guide

Functional state isolation: one bloc, one job

Each bloc owns exactly one slice of state and never writes to another bloc's state.



✕ CartBloc calls todoBloc.add(…) ✕ TodoBloc reads authBloc.state in a handler

✓ Widgets wire them: BlocListener on AuthBloc adds CartCleared to CartBloc
✓ Or: CartBloc subscribes to authBloc.stream in its constructor

- Cross-bloc communication goes through events or stream subscriptions, never direct state mutation.
- Prefer wiring in the widget layer (BlocListener): dependencies are visible in one place and the blocs stay independent.
- If a bloc subscribes to another, cancel the subscription in close().

cart_bloc.dart

```dart
// Option A: wire in the widget tree (preferred)
BlocListener<AuthBloc, AuthState>(
  listener: (context, state) {
    if (state is Unauthenticated) {
      context.read<CartBloc>().add(const CartCleared());
    }
  },
  child: const HomePage(),
)

// Option B: subscribe inside the bloc
class CartBloc extends Bloc<CartEvent, CartState> {
  CartBloc(this._authBloc) : super(const CartState()) {
    on<CartCleared>((e, emit) => emit(const CartState()));
    _authSub = _authBloc.stream.listen((auth) {
      if (auth is Unauthenticated) add(const CartCleared());
    });
  }
  final AuthBloc _authBloc;
  late final StreamSubscription<AuthState> _authSub;

  @override
  Future<void> close() {
    _authSub.cancel();        // always clean up
    return super.close();
  }
}
```

Sources: [B10] bloclibrary.dev, Architecture

BlocObserver: one place to watch everything

WHAT

A global hook that is called for every bloc: creation, events, state changes, transitions, errors and close. Set once with Bloc.observer = ….

onChange vs onTransition

onChange fires for Cubits and Blocs (current → next state). onTransition fires only for Blocs and includes the event that caused the change.

USE IT FOR

Debug logging in development, crash reporting and analytics in production, or asserting on transitions in integration tests. It is the traceability Cubit alone cannot give you.

app_bloc_observer.dart

```dart
class AppBlocObserver extends BlocObserver {
  const AppBlocObserver();

  @override
  void onCreate(BlocBase<dynamic> bloc) {
    super.onCreate(bloc);
    log('created ${bloc.runtimeType}');
  }

  @override
  void onTransition(Bloc<dynamic, dynamic> bloc,
      Transition<dynamic, dynamic> transition) {
    super.onTransition(bloc, transition);
    log('${bloc.runtimeType}: '
        '${transition.event} -> ${transition.nextState}');
  }

  @override
  void onError(BlocBase<dynamic> bloc,
      Object error, StackTrace stackTrace) {
    log('${bloc.runtimeType} error: $error');
    super.onError(bloc, error, stackTrace);
  }
}

// main.dart
Bloc.observer = const AppBlocObserver();
```

Sources: [B1] bloclibrary.dev, BlocObserver

PART 8

Testing blocs

Unit tests with bloc_test: given a bloc and events, assert the exact sequence of states. Then widget tests with a fake bloc.

blocTest: build, act, expect

- build creates the bloc under test, usually with a fake or mocked repository.
- act sends events (or calls cubit methods).
- expect is the list of states the bloc must emit, in order. The initial state is not included.
- mocktail stubs the repository: when(() => repo.fetchAll()).thenAnswer(...).
- States need value equality for expect to match. Use equatable, freezed or hand-written ==.

todo_bloc_test.dart

```dart
import 'package:bloc_test/bloc_test.dart';
import 'package:mocktail/mocktail.dart';

class MockTodoRepository extends Mock
    implements TodoRepository {}

void main() {
  late MockTodoRepository repo;
  final todos = [const Todo(id: '1', title: 'Milk')];

  setUp(() {
    repo = MockTodoRepository();
    when(() => repo.fetchAll())
        .thenAnswer((_) async => todos);
  });

  blocTest<TodoBloc, TodoState>(
    'emits [loading, success] when load succeeds',
    build: () => TodoBloc(repo),
    act: (bloc) => bloc.add(const TodoLoadRequested()),
    expect: () => [
      const TodoLoading(),
      TodoSuccess(todos),
    ],
  );
}
```

Sources: [B11] pub.dev/packages/bloc_test

Error paths, seed and verify

E — Test the failure path

Stub the repository to throw and expect TodoFailure. The unhappy path is where most bugs live.

S — seed: start mid-flow

Put the bloc in a given state before act, e.g. TodoSuccess with two items, then test remove without loading first.

V — verify: check side effects

Assert that the repository was called with the right arguments and the right number of times.

todo_bloc_test.dart (continued)

```dart
blocTest<TodoBloc, TodoState>(
  'emits [loading, failure] when load throws',
  build: () {
    when(() => repo.fetchAll())
        .thenThrow(Exception('network'));
    return TodoBloc(repo);
  },
  act: (bloc) => bloc.add(const TodoLoadRequested()),
  expect: () => [
    const TodoLoading(),
    isA<TodoFailure>(),      // matcher, not equality
  ],
);

blocTest<TodoBloc, TodoState>(
  'removes a todo from an existing list',
  build: () {
    when(() => repo.delete('1')).thenAnswer((_) async {});
    return TodoBloc(repo);
  },
  seed: () => TodoSuccess(todos),          // start here
  act: (bloc) => bloc.add(const TodoRemoved('1')),
  expect: () => [const TodoSuccess([])],
  verify: (_) => verify(() => repo.delete('1')).called(1),
);
```

Sources: [B11] pub.dev/packages/bloc_test

Widget tests with a fake bloc

- MockBloc from bloc_test gives you a bloc whose state you control. No repository, no network.
- Stub the state with when(() => bloc.state).thenReturn(…), wrap the widget in BlocProvider.value, pump, and assert on what is on screen.
- This tests the presentation layer alone: given this state, is this widget shown?
- whenListen lets you emit a sequence of states and test BlocListener side effects such as snackbars.

todo_page_test.dart

```dart
class MockTodoBloc extends MockBloc<TodoEvent, TodoState>
    implements TodoBloc {}

void main() {
  late MockTodoBloc bloc;
  setUp(() => bloc = MockTodoBloc());

  testWidgets('shows todos when state is success',
      (tester) async {
    when(() => bloc.state).thenReturn(
      TodoSuccess([const Todo(id: '1', title: 'Milk')]),
    );

    await tester.pumpWidget(
      MaterialApp(
        home: BlocProvider<TodoBloc>.value(
          value: bloc,
          child: const TodoPage(),
        ),
      ),
    );

    expect(find.text('Milk'), findsOneWidget);
    expect(find.byType(CircularProgressIndicator),
        findsNothing);
  });
}
```

Sources: [B11] pub.dev/packages/bloc_test

PART 9

Riverpod or Bloc?

The same concepts side by side, a decision guide, and the key takeaways of the lecture.

COMPARISON

The same ideas, two vocabularies

Concept	Riverpod 3.x	Bloc / Cubit
Holds state + methods	`Notifier / AsyncNotifier`	`Cubit / Bloc`
Change state	`state = newValue`	`emit(newValue)`
Trigger a change	`ref.read(p.notifier).method()`	`context.read<B>().add(Event)`
Rebuild on change	`ref.watch(provider)`	`BlocBuilder / context.watch`
Side effects	`ref.listen`	`BlocListener`
Watch one field	`select((s) => s.x)`	`BlocSelector / context.select`
Async loading	`AsyncValue (built in)`	`status enum or sealed states (by hand)`
Provide to the tree	`ProviderScope (one, at root)`	`BlocProvider / RepositoryProvider`
Inject a fake in tests	`overrides: […]`	`constructor argument / MockBloc`
Concurrency control	`manual (keepAlive, mounted)`	`event transformers`
Global logging	`ProviderObserver`	`BlocObserver (with events)`

Sources: [19] Riverpod for Provider users, [B1] bloclibrary.dev

SUMMARY

Key takeaways

1 Both separate logic from UI

Riverpod via providers and Notifiers, Bloc via events and states. In both, widgets only watch state and call methods or add events.

2 Cubit first, Bloc when needed

Cubit is a Notifier with emit(). Move to Bloc when you need named events, transition logs or transformers.

3 Streams are the engine

Bloc is a StreamController with rules. debounce, switchMap and emit.forEach are stream operators applied to events.

4 Transformers tame concurrency

restartable for search, droppable for submit buttons, sequential for ordered writes, concurrent for independent reads.

5 Clean layers keep blocs testable

Blocs depend on domain interfaces; repositories implement them in the data layer. Fake the repository, test the bloc.

6 Choose by team and traceability

Riverpod: less code, reactive graph, AsyncValue. Bloc: explicit events, global observer, strong conventions for large teams.

Sources: flutter_riverpod 3.4.3, flutter_bloc 9.x, Oct 2026